

P3665R0

Vertical Text Processing

Published Proposal, 2025-04-01

This version:

<https://wg21.link/P3665R0>

Authors:

[Jeremy Rifkin](#)

Levi de koning

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/jeremy-rifkin/vertical-text-processing/blob/main/vertical-text-processing.bs>

Abstract

While text is most often written horizontally, many writing systems can be written vertically. Yet, C++ offers no standard support for such text. This paper outlines a proposal to rectify the situation.

Table of Contents

- 1 Introduction**
- 2 Motivation**
- 3 Recommended Solution**
- 4 Why This Isn't a Tooling Problem**
- 5 Design**
- 6 Switching out of vertical processing mode**
- 7 Implementability**
- 8 Improving WG21 Papers**
- 9 Proposed Wording**
- 10 Annex A: Supplemental figs**

References

Normative References

Informative References

§ 1. Introduction

Many writing systems, notably a handful of East Asian scripts, may be oriented either horizontally or vertically. While code is generally written left to right and top to bottom, there is no inherent reason this must be the case. In fact, in this paper we

will show that vertical writing of code has numerous benefits over traditional horizontal code.

In this paper we propose support for vertical text in C++:

```
#define __vertical_processing
}      it
      ne
      tm
      p
r }    ftsl
e      oyua
t      rpm t
u      (e(e
r      scns<
n      uoatt
      mnm dy
s      se:p
u      +t :e
m      = ss n
;      atpa
      iudam
      tt:ne
      eo:<
m      &rTT
;      e , ,
      im
      toEs
      ev>t
      me d
      _a :
      :cr:
      ors
      an)i
      rs z
      rt{e
      ) _ _
      t t
      {<
      T E
      > >

      s
      u
      m
      {
      }
      ;
```

§ 2. Motivation

In addition to accessibility and friendliness for natural writing styles of all cultures, vertical writing of code allows for better use of modern computer monitors. Over the last few decades, monitors have definitively trended to get wider but not so much taller (fig a). We observe that programmers typically limit their line length (when written left-to-right) at fixed character limits such as 80, 100, or 120. Meanwhile, the vertical length of a source file typically has no hard limits. Given recent trends and accepted practices with regards to line length limitations, it's clearly beneficial to write code vertically with line length limitations applying on the vertical access so that code may benefit from today's wide screen monitors, as well as future wider screen monitors.

Fig A: An ultra-widescreen monitor, circa 2025

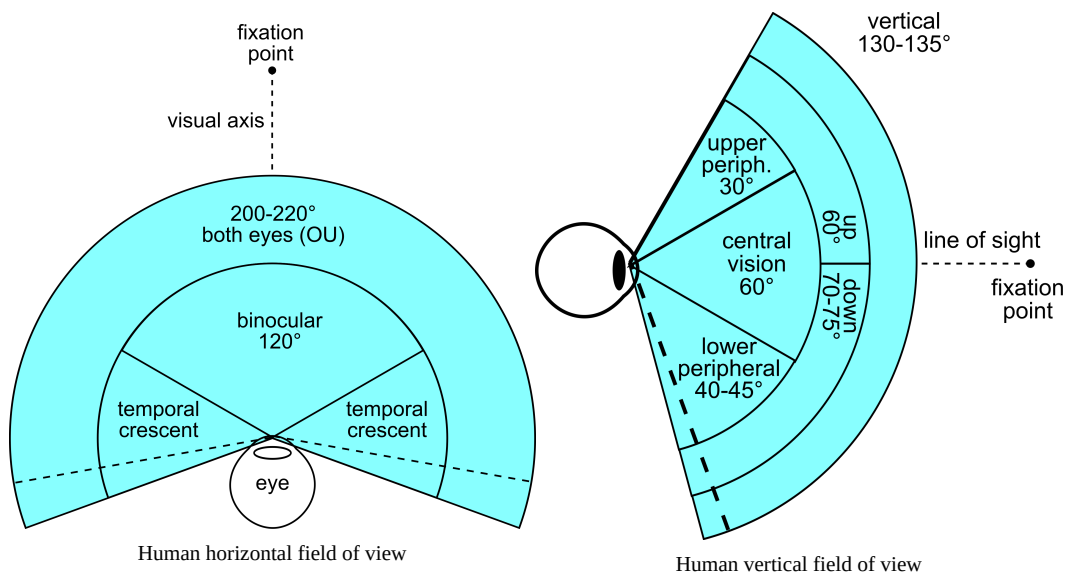


Image Source: [RTINGS.com](https://www.rtings.com)

Because humans have a much narrower vertical field of view compared to horizontal (fig b), it's not a satisfactory solution to rotate monitors vertically. Additionally, with today's ultra-wide monitors, vertical orientation would not work on most desks and would lead to unergonomic craning of the neck.

As such, by taking advantage of vertical text on ultrawide monitors, we can read more characters before we need to move our head reducing excessive muscle movement during programming.

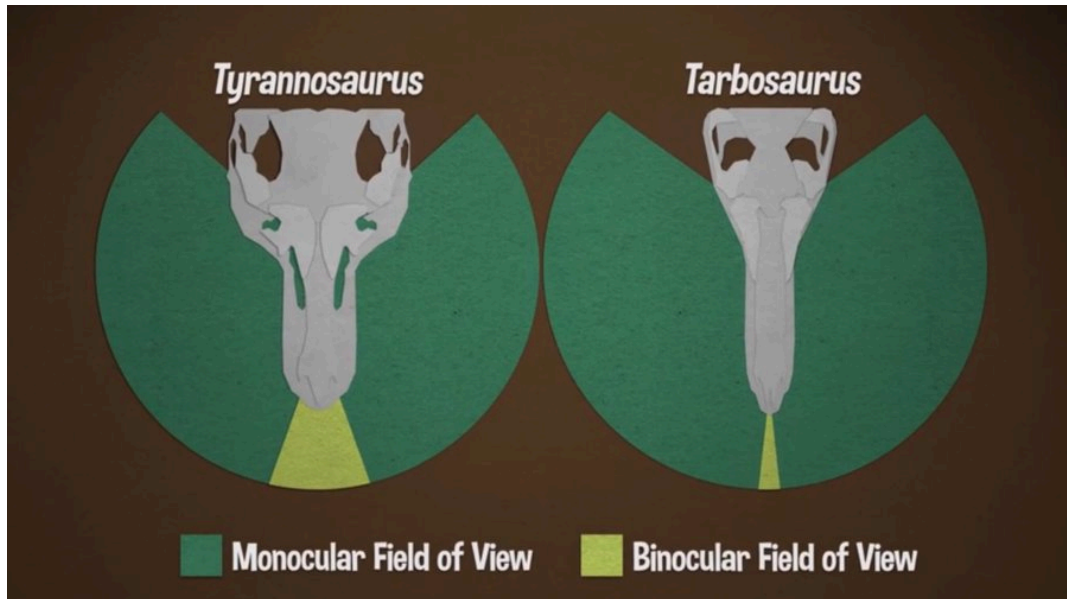
Fig B: Human eye field of view



Source: [Wikipedia](https://en.wikipedia.org/wiki/Field_of_vision) and [Wikipedia](https://en.wikipedia.org/wiki/Field_of_vision), contributed by Zyxxwv99

Humans are not alone in this regard, many animals have much wider horizontal field of view compared to vertical. For example, the Tyrannosaurus rex as with other tyrannosaurids and theropods feature an even greater total field of view:

Fig C: Field of view for Tyrannosaurus and Tarbosaurus



Source: [Edaphosaurus](#)

Sacrificing narrower binocular field of view for more total field of view is a trait attributed to their predatory nature and evidenced by the eye placement in the skulls of these dinosaurs:

Fig D: A *Tyrannosaurus rex* pictured making the famous "pogchamp" face



Source: [Binocular Vision in Theropod Dinosaurs](#)

NOTE: Despite possibly being fluffy and covered by feathers, do not pet these animals.

As such, we believe the *Tyrannosaurus rex* as well as other tyrannosaurids and theropods are well-equipped for modern wide-screen monitors and would likely program quite effectively in the vertical orientation on such devices. We are currently seeking funding for exploring this hypothesis.

See also: [§ 10 Annex A: Supplemental figs](#)

§ 3. Recommended Solution

We propose adding support to the C++ preprocessor for vertical text via the preprocessor definition, `__vertical_processing`. For example:

```
#define __vertical_processing
} i
n
r t
e
t s
u q
r u
n a
r
n e
u (
m i
n
* t

n n
u u
m m
; )

{
```

Here's a slightly more involved example:

```
#define __vertical_processing
} i ###
n i i i
t n n n
c c c
) s } w i s m l l l
; t h n t a u u u
d i t d i d d d
: l : n e e e
) s " : v e x : (
t S p e ; v ) < < <
d u r c ( e p v a
: m i . s c { r e l
s 0 v v : : n p t t i c g
t , e e r t u d o n t o
d c c a { l s : r t o r
: . . n } n h : < > r i
: e b g " ( _ c i > t
p n e e , b i n h
l d g s a n t m
u ( i : c > >
s ) n : k >
< , ( f ( > v
i ) o x e
n , l ) x c
t d ; ) ;
>
( l {
) e
f
t
(
```

As one would expect, reading such code becomes perfectly natural after just a few minutes of looking at it.

To examine a real world example, here is Erik Rigtorp's SPSCQueue translated to vertical text [\[rigtorp\]](#). Even at a glance it is much easier to read (not that the code is hard to read when laid out traditionally, it is in fact quite nicely written code, made even easier by this paper). Be sure to use an ultrawide wide monitor for the best viewing experience:

```
#define __vertical_processing
} }
; # # # p # # # p
/ aaaa/// nAlAfTsi s/ nsl sfi R } R } R }
/ llll/// dls l iv t/ dts tdv I I I
iiii ileld*za a ia e aea G rG r} i sG r} ias aa s
n ggggcrA fo oeset tP ft tft T eT e f tT e ful su t
a nnnnoel c cfl_e ia i si e 0 t0 t d d0 a n to st a
m aaaa hai a ai ot: cd c tc_: R uR u i (:R d e (ot eo t|
e ssssdg t tnt d d _ P rP r fd :P I xn s r i
s (((rIn o oesc ci c :cc _ n_ n fi p_ d ten_ tc c|
p kkkked r rd_a on o :op N N f tN x Rxe[ (o _
a CCCcnxt (;p ng n hnp 0 rw0 s +f r0 _ etxr "wn a
c aaaaacCo a a_ a s s as_ D erD t = dD . aRteCrs s
e ccccy a l l_ c tt t rtl I aiI a < iI s deRaa it s|
h h h h c c l l h i eo e dei S dtS t c fS t Ia ednt e|
r eeeetha o oa t x x wxb C IeC i a0 fC o ddaI er r|
i LLLLrec c cs y pa p ap_ A dIA c p) _A r xId doIe t:
g iiii a_h a a_ _ rv r rrh R xdR _ a tR e dIx nda (
t nnnnf e t tc ; o e a D _xD c c{ D ( =xd lxd "s
o eeeefa o op si s _sr . _ a i d n x+y_ ITt|
r SSSSinl r rp id i did s l.b s t is e 0 = . d d|
p iiii cdi _ _ z z ez w i olo t y fi x ; == kclxm:
zzzz n ; a ef e sea z aoo < _ fz t Pao u:
eeee we [t _a _ t_r e dal s ; e R crala=s|
))) r [t tl t rte _ (d i = _ e aedld ts
is nr s u _ t s(e z t a pad (r _
ssss ti oi ke k cki tsm e rw d adipsebn
itit ez _b P C tCn c dtp _ ers I cInota eo
zdzd Ie uu as a iat a :dt t aii d idgpdd t
e:e: d nt dh c vce p ::y > dtz x tx| (:Inh
_:_: xi ie da h ehr a m:( ( Iee , y .):dor
tata Cn q) ir e _ef c em) d dI( _+~ mxto
t t a u ni L iLe i me i xd) s ) Tae_hw
woro co e& gn i nir t omc f _x t 1(fm.r_
rmem hr _& g n tne y roo f . _c d {; ) tolod
iaai ed a = e een ( yrn ) l.o : ;erowe
tcdc _e d _ b S rSc ) _ys ; oln : rya s
e<I< r d_ (e i fie o_t aos m _ddt
Isds i rh kt z ez_ c ro dat e fo(er
dixi st ea e res o drn (d m rrrssu
xzCz o ss Ae e i n edo s(n o odttc
Cea e u s_ ce = n=z s ree tso r nedrt
a_c_ sa ]c hn c e t _rx dte y tr:ui
cth t ev ]p e 6 e a_c :dx _ ( _:cb
h>e> d o ;p Ls 4 _ n cae ::c o )amt|
e _ i _ il ; s o qcp m:e r ceie
_r w td a no i e uqt emp d hqmb<
e=r o t et z x iu met e auoltT
=a i f t Ss e c ri{ om r sire>
d0t ra i _ ; e er ro{ _ ry":
0I;e el i z p )e y r r re_):
;d I ds b ea t ;) _y e e)o;v
x d ue u n { = ro e t r a
_ x c t -d { = ro e u!d l
_ es e = dr a r=e u
= h ( 1 a r e ed s n r e
= ta n ) d e re e er_ ,
```

```

{   h r   o   j   t   _ r   )   d e r
0 {   e i   _   / a   u   a _   ;   a e
} 0   n   u   c   r   c a   a d l
; } a g   n   s e   n   q c   I a
;   m   i   i n   u q   n d x
   o   q   z t   c   i u   o x e
   u   u   e   a   r i   n d
   n   e   o a   p   e r   - & )
   t   _   f l   a   ) e   n & ;
           a   ( l   c   ; )   u
           d   T o   i   l
           f   d   ) c   t   -   l
           r   a   y   p
           e   + t   -   t
           s   i   r
           s   1 o   -   "
           )   ; n   )
           s   s   1   ;
           ;   }

```

§ 4. Why This Isn't a Tooling Problem

One could imagine tooling support for vertical code where the tooling saves vertically written text to disk equivalently to if it were written horizontally. Naturally, there are no rules stating that characters on disk are to be laid out horizontally by a text editor. However, this convention has been so deeply ingrained into software since the beginning of the universe (January 1, 1970), that it is de-facto a law of computer software.

We find any editor tooling solutions undesirable for a couple of important reasons:

Firstly, despite decades of innovation in the text editor domain, no tooling supports this obvious feature. As such, we don't anticipate any native tooling support any time soon. Naturally, even if some tools added support, there will always be software lagging behind in support. We view this as imputus for WG21 to pick up the torch and lead industry adoption.

Additionally, this opens the door for code to be laid out in radically different ways for various users depending on how they are reading the code. This is the same problem encountered by "hard tabs," which are rendered by default as 4 characters wide on good editors and 8 characters wide on some other editors (that's as many characters as bits in a `char`!).

§ 5. Design

Unfortunately, because the decision was made by Bjarne Stroustrup to have C++ compilers interpret source code linearly in a file, thus corresponding to horizontally as presented by editors, the directive used to change processing must be horizontal as well. We recommend implementations facilitate options and command-line parameters for this in addition to the standard-sanctioned approach.

§ 6. Switching out of vertical processing mode

The syntax begs the question: What about `#undef __vertical_processing`? The solution is simply elegant once the preprocessor / lexer encounters `#undef __vertical_processing` we immediatly switch back to the legacy mode of processing. Naturally, this directive must also be horizontal thanks to reasonable but unfortunate decisions made early on in the design of C++.

For example:

```
#define __vertical_processing
i
n
t

s
q
u
a
r
e
(
i
n
t

n
u
m
)

{
#undef __vertical_processing
return num * num;
}
```

§ 7. Implementability

The transformation between traditional horizontal and vertical text is firmly rooted in mathematics, which lends this to a straightforward implementation by adding a new translation phase between phases 1 and 2. In translation phase 1.5, a compiler may trivially identify vertical processing sections within a file and apply the following transformation:

For ease of implementation, we require the exact spelling `#define __vertical_processing` with no additional whitespace in the middle and we also require it to appear on its own line.

§ 8. Improving WG21 Papers

There's no reason this should be limited to source code, we can apply this same technology to improve papers submitted to WG21:

Improve this paper

§ 9. Proposed Wording

Proposed wording relative to [\[N5008\]](#):

Add a new list item numbered 1.5 after [lex.phases] paragraph 1 list item 1:

1.5: A physical source line containing only the sequence `#define __vertical_processing`, optionally with preceeding or trailing whitespace characters, causes the implementation to switch to *vertical text processing mode* for subsequent physical source lines. A physical source line containing only the sequence `#undef __vertical_processing`, optionally with preceeding or trailing whitespace characters, causes the implementation to switch out of *vertical text processing mode*. Sequences of physical source lines interpreted in *vertical text processing mode* shall be transformed as if by the following transformation:

$$B_{ij} = \sum_{p=1}^M \sum_{q=1}^N A_{qp}^T \frac{1}{2\pi i} \oint_{|z|=1} z^{p-(M+1-j)-1} dz \frac{1}{2\pi i} \oint_{|w|=1} w^{q-i-1} dw$$

[Note: #undef __vertical_processing is not required to appear in a source file after a #define __vertical_processing directive. — end note]

§ 10. Annex A: Supplemental figs

Fig E and G:



Source: [freerange](#)

§ References

§ Normative References

[N5008]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 15 March 2025. URL: <https://wg21.link/n5008>

§ Informative References

[BINOCULARVISIONINTHEROPODS]

Kent A. Stevens. *Binocular Vision in Theropod Dinosaurs*. URL: https://www.researchgate.net/profile/Kent-Stevens/publication/228671730_Binocular_vision_in_theropod_dinosaurs/links/0f317533da394affe6000000/Binocular-vision-in-theropod-dinosaurs.pdf

[RIGTORP]

Erik Rigtorp. *SPSCQueue*. URL: <https://github.com/rigtorp/SPSCQueue>